

Bài 16: Map Framework trong Java

Map là interface trong `java.util`, lưu dữ liệu dưới dạng cặp **key - value**.

- Mỗi **key** là duy nhất và ánh xạ tới đúng một **value**.
- Map** không kế thừa từ interface **Collection**.

1. Các cài đặt của Map

HashMap

- Cài đặt **Map** phổ biến nhất.
- Không đảm bảo thứ tự các phần tử.
- Cho phép một **key** là **null** và nhiều **value** là **null**.
- Không đồng bộ (không thread-safe).

LinkedHashMap

- Kế thừa từ **HashMap**.
- Giữ thứ tự thêm vào nhờ danh sách liên kết đôi.
- Cho phép một **key** là **null** và nhiều **value** là **null**.
- Chậm hơn **HashMap** một chút vì phải duy trì thứ tự.

TreeMap

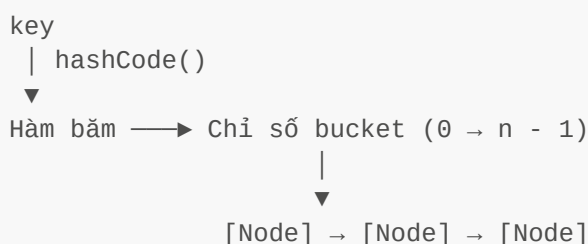
- Lưu key theo thứ tự đã sắp xếp (thứ tự tự nhiên hoặc **Comparator**).
- Không cho phép **key** là **null** khi dùng thứ tự tự nhiên.
- Chậm hơn **HashMap** vì thao tác cần duy trì cây sắp xếp.

Hashtable

- Class cũ từ Java 1.0.
- Đồng bộ và an toàn đa luồng.
- Không cho phép **key** hoặc **value** là **null**.
- Thường ưu tiên **HashMap**; khi cần đồng bộ có thể dùng **ConcurrentHashMap**.

2. HashMap hoạt động bên trong

- HashMap** dùng mảng các bucket (thùng chứa) để lưu entry.
- hashCode()** của key được dùng để tính vị trí bucket.
- Nếu nhiều key cùng bucket (collision), các entry được liên kết với nhau.
- Từ Java 8, khi collision nhiều, bucket có thể chuyển từ linked list sang cây để truy xuất tốt hơn.



Khi lấy giá trị bằng `get(key)`, **HashMap** tính lại hash của key, tìm bucket tương ứng rồi so sánh key bằng `equals()`.

3. Bảng so sánh

Tiêu chí	HashMap	LinkedHashMap	TreeMap	Hashtable
Thứ tự	Không có thứ tự	Thứ tự thêm vào	Sắp xếp theo key	Không có thứ tự
Key <code>null</code>	Cho phép (1 key)	Cho phép (1 key)	Không cho phép*	Không cho phép
Value <code>null</code>	Cho phép	Cho phép	Cho phép	Không cho phép
An toàn đa luồng	Không	Không	Không	Có (đồng bộ)
Hiệu năng	Nhanh nhất trung bình: $O(1)$	Chậm hơn HashMap chút ít	$O(\log n)$	Chậm hơn do đồng bộ
Cấu trúc dữ liệu bên dưới	Mảng bucket + node/cây	Hash table + doubly linked list	Cây đỏ-đen	Hash table + linked list
Class	<code>java.util.HashMap</code>	<code>java.util.LinkedHashMap</code>	<code>java.util.TreeMap</code>	<code>java.util.Hashtable</code>

*`TreeMap` có thể nhận key `null` nếu `Comparator` được cung cấp và comparator đó xử lý được `null`.

4. Khi nào dùng loại nào?

- Dùng `HashMap` khi không quan tâm thứ tự và cần hiệu năng tốt.
- Dùng `LinkedHashMap` khi cần giữ thứ tự thêm vào.
- Dùng `TreeMap` khi cần key được sắp xếp.
- Tránh dùng `Hashtable` trong code mới; cân nhắc `ConcurrentHashMap` cho môi trường đa luồng.

5. Ví dụ HashMap

```
import java.util.HashMap;
import java.util.Map;

Map<String, Integer> map = new HashMap<>();

map.put("A", 10);           // Thêm hoặc cập nhật value của key A
map.put("B", 20);
map.put(null, 30);         // HashMap cho phép một key null
map.put("A", 40);          // Cập nhật: key A có value mới là 40

System.out.println(map.get("A"));           // 40
System.out.println(map.containsKey("B"));   // true
System.out.println(map.remove("B"));        // 20
System.out.println(map.size());             // 2
```

6. Phương thức thường dùng

Phương thức	Mô tả
<code>put(key, value)</code>	Thêm hoặc cập nhật phần tử
<code>get(key)</code>	Lấy value theo key
<code>remove(key)</code>	Xóa phần tử theo key
<code>containsKey(key)</code>	Kiểm tra key tồn tại
<code>containsValue(value)</code>	Kiểm tra value tồn tại
<code>size()</code>	Số phần tử trong map
<code>keySet()</code>	Lấy tập hợp các key
<code>values()</code>	Lấy tập hợp các value
<code>entrySet()</code>	Lấy tập hợp các cặp key - value

7. Ghi nhớ

- `key` nên là kiểu không thay đổi (immutable), ví dụ `String`, để hash không bị thay đổi sau khi thêm vào map.
- Nếu tự tạo class làm key, cần ghi đè đúng `hashCode()` và `equals()`.
- Chọn implementation dựa trên yêu cầu về thứ tự, tốc độ và đa luồng.